

Informed Search

CHAPTER 3

Informed (Heuristic) Search

□ Heuristic – problem-specific knowledge

- Finds solutions more efficiently

□ New terms

- $g(n)$: cost from initial state to the state at node n
- $h(n)$: estimated cost from state at node n to the closest goal
 - Depends only in the state at node n
 - $h(n) = 0$, if n is a goal node
- $f(n)$: evaluation function; cost estimate from the initial state to the closest goal state through the state at node n

□ Genre of Best First Search (BFS)

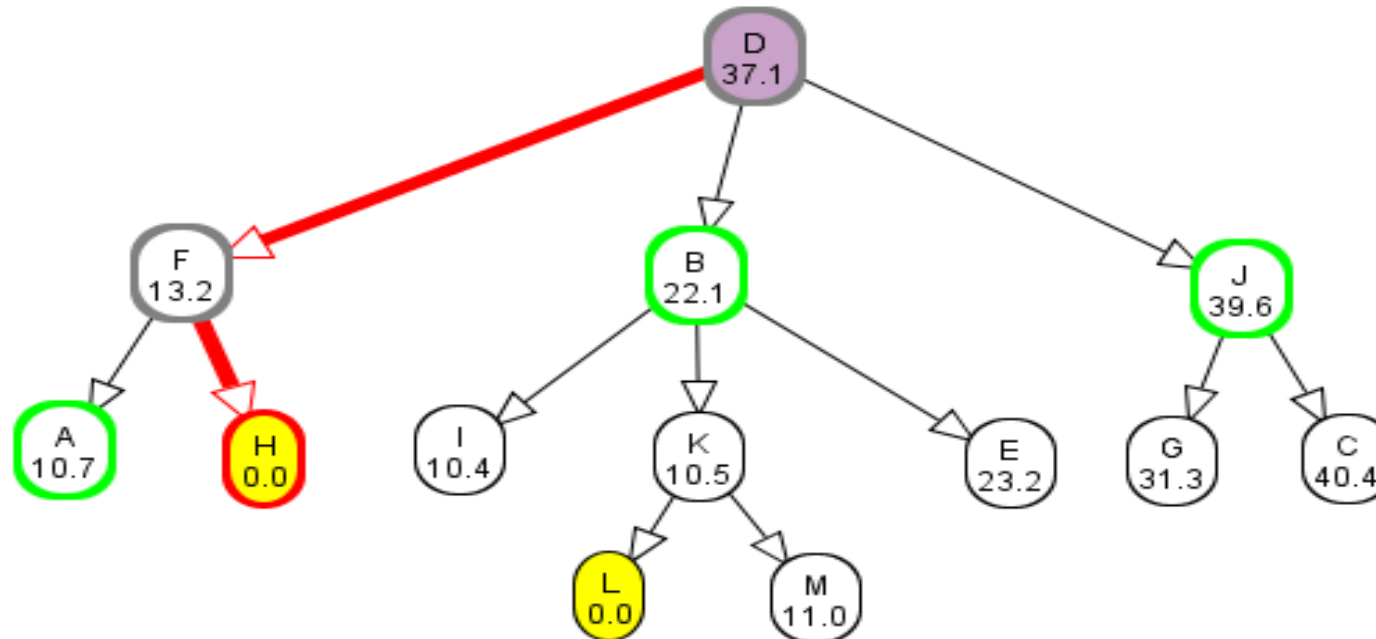
- Greedy BFS
- A* search

Greedy Best First Search (GBFS)

❑ Expand the most “desirable” node

- Desirability is measured through the evaluation function $f(n)$ and here $f(n) = h(n)$

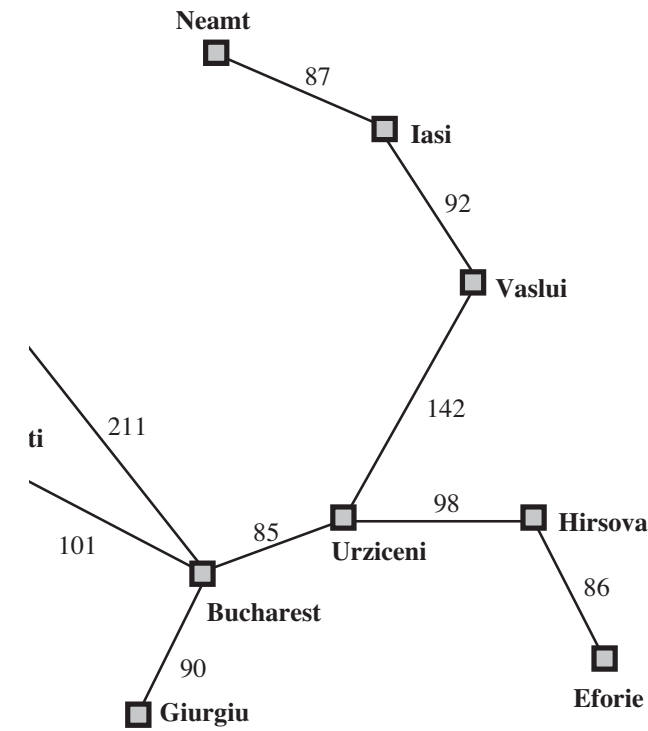
❑ Implementation: priority queue based on $h(n)$



GBFS - Analysis

- ❑ Completeness: No, can get stuck in loops
 - Can be made complete with repeated state checking
- ❑ Optimality: No
- ❑ Time Complexity: $O(b^m)$
- ❑ Space Complexity: $O(b^m)$
 - Keeps all nodes in memory.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



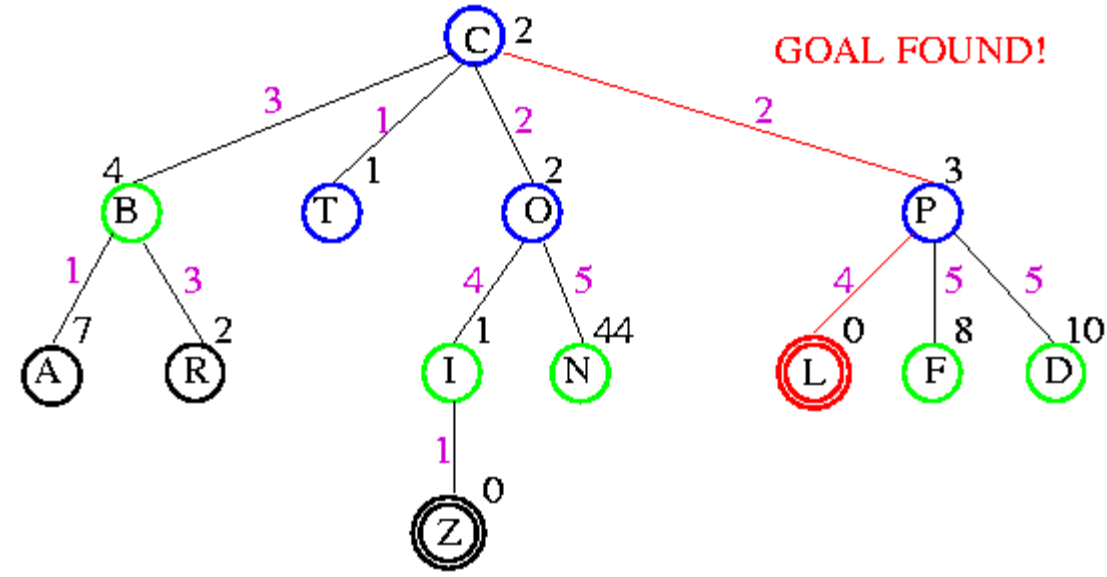
A* search

- ❑ Idea: avoid expanding paths that are already expensive.
- ❑ Implementation: priority queue based on the evaluation function $f(n)$

$$f(n) = g(n) + h(n)$$

- $g(n)$: cost so far to reach the node n
 - $h(n)$: estimated cost to goal from n
 - $f(n)$: estimated total cost of path through n to goal.
- ❑ Identical to uniform cost search(UCS) except that we use $g + h$ instead of only g

A* - Example



Open List = L (6+0=6), B (3+4=7)
I (6+1=7), F (7+8=15), D (7+10=17), N (7+44=51)

A* search - Analysis

- ❑ Completeness: Yes, unless there are infinitely many nodes with $f \leq f(G)$
- ❑ Optimality:???

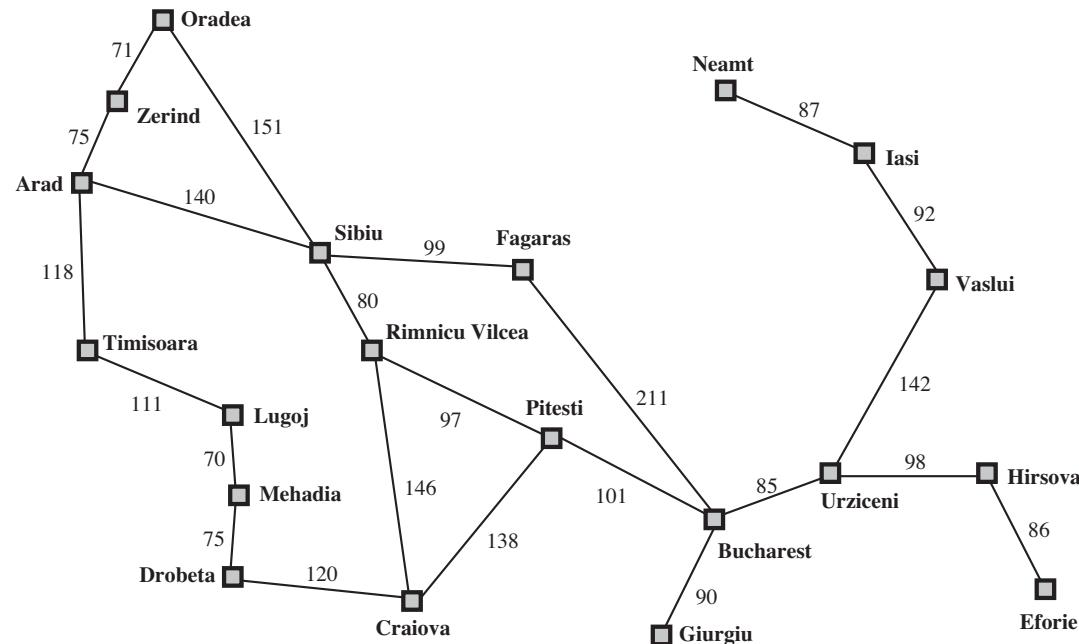
Admissible Heuristic

□ A* uses an admissible heuristic $h(n)$

$$h(n) \leq h^*(n), \forall n$$

○ $h^*(n)$ – is the true cost to the goal node from n

○ $h(n) \geq 0, \forall n$



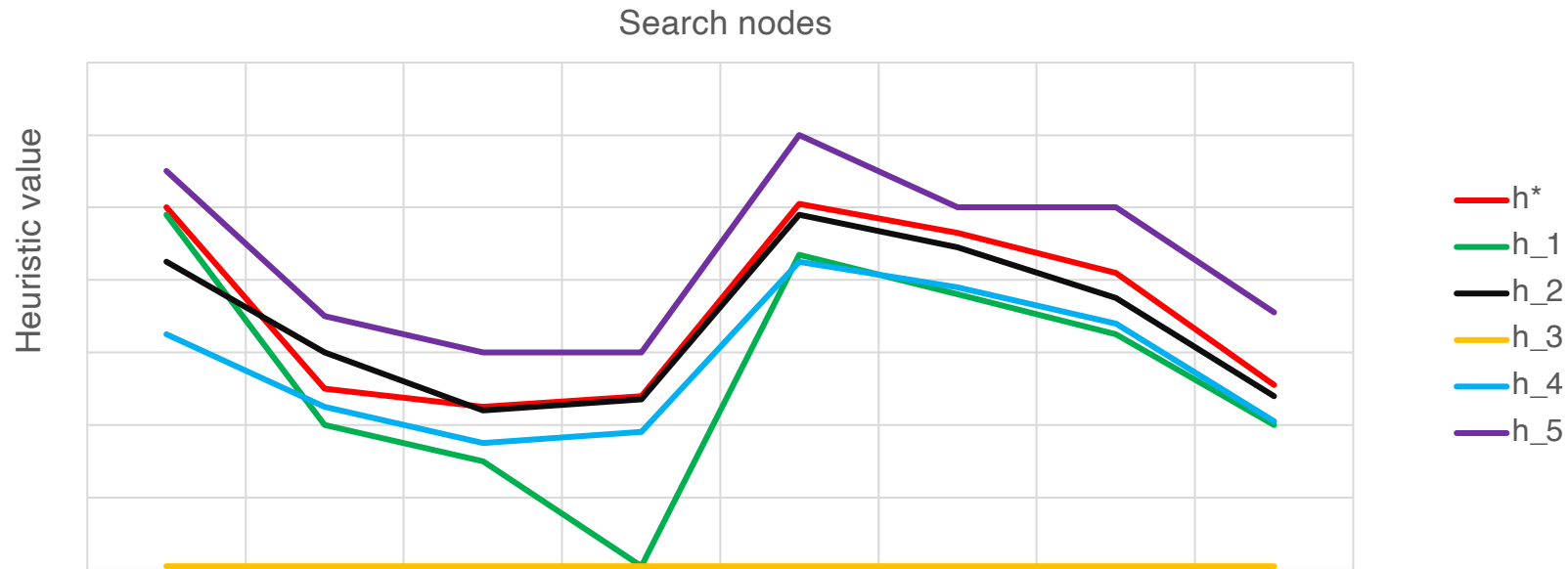
Admissible Heuristic

□ A^* uses an admissible heuristic $h(n)$

$$h(n) \leq h^*(n), \forall n$$

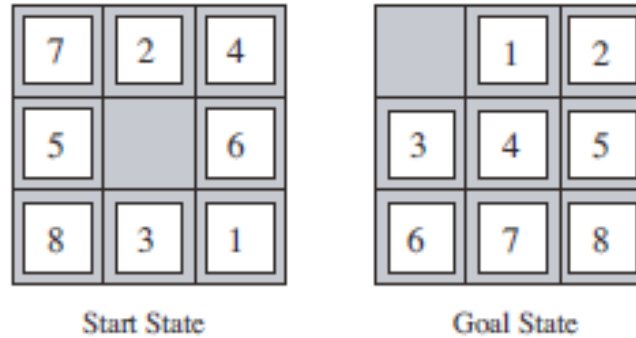
○ $h^*(n)$ – is the true cost to the goal node from n

○ $h(n) \geq 0, \forall n$



Example Heuristic Functions (1)

❑ 8-puzzle problem



❑ Examples

- Number of misplaced tiles
- Total Manhattan distance

Example Heuristic Functions (2)

□ Romania Tourist Problem

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

□ Examples

- Straight line distance – never overestimates the actual road distance

A* search - optimality

□ Proof by contradiction

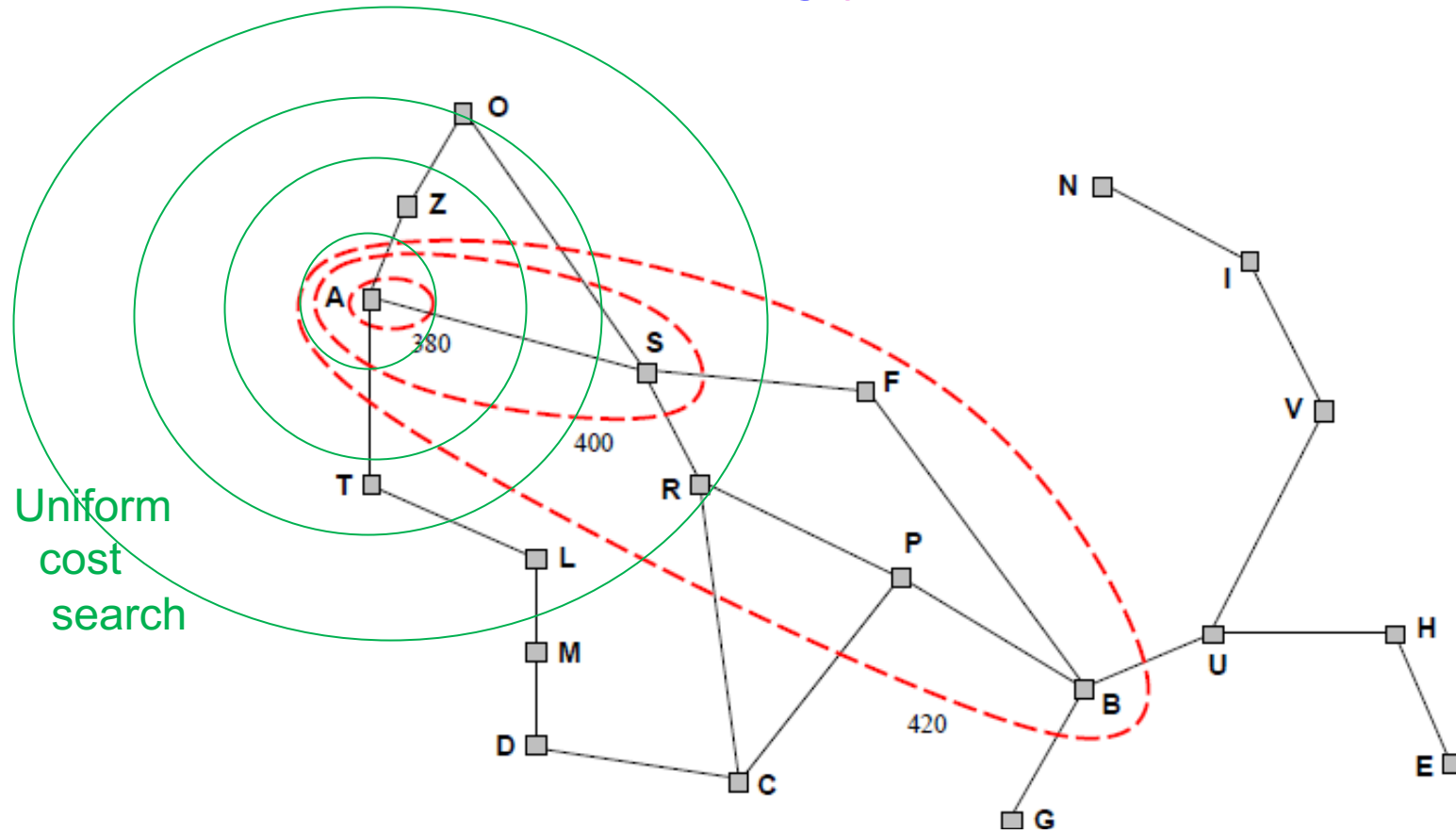
- Let N be the goal node A* outputs and suppose there is another goal node N' .
Then $g(N) \leq g(N')$
- Assume to the contrary $g(N') < g(N)$
- When we picked N for expansion, either N' or an ancestor of N' - N'' must have been on the queue. Since we picked N for expansion
- $f(N) \leq f(N'')$ implies $g(N) + h(N) \leq g(N'') + h(N'')$
- $g(N) \leq g(N'') + h(N'')$ (1) For a goal node $h(N) = 0$
- $g(N') = g(N'') + \text{dist}(N', N'')$, $h(N'') \leq h^*(N'') = \text{dist}(N', N'')$
- So $g(N') \geq g(N'') + h(N'')$ (2)
- From (1) and (2) $g(N) \leq g(N')$ - contradiction

A* search - Analysis

- ❑ Completeness: Yes, unless there are infinitely many nodes with $f \leq f(G)$
- ❑ Optimality: Yes
- ❑ Space Complexity: Keeps all nodes in memory
- ❑ Time Complexity: exponential in [relative error in h * length of the solution]
 - A* expands all nodes with $f < C^*$
 - A* expands some nodes with $f = C^*$
 - A* expands no nodes with $f > C^*$

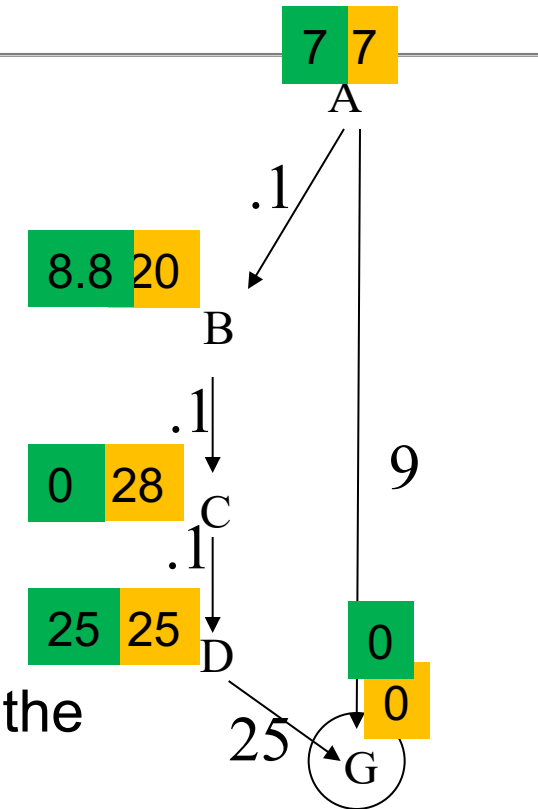
Visualizing A* Search

□ A* expands nodes of increasing f value



Admissibility, Monotonicity, Pathmax Correction

- ❑ Is orange h admissible?
- ❑ Is green h admissible?
- ❑ Does $f(C)$ make sense?
 - $f(B) = .1 + 8.8 = 8.9$
 - $f(C) = .2 + 0 = 0.2$
- ❑ Path cost estimate reduces
 - This doesn't make sense since we are reducing the estimate of the actual cost of the path
- ❑ To make $f(.)$ monotonic along a path, we say
$$f(n) = \max(f(\text{parent}), g(n) + h(n))$$
 - Also referred to as Pathmax correction



Monotonic Heuristic

- Consistent Heuristic

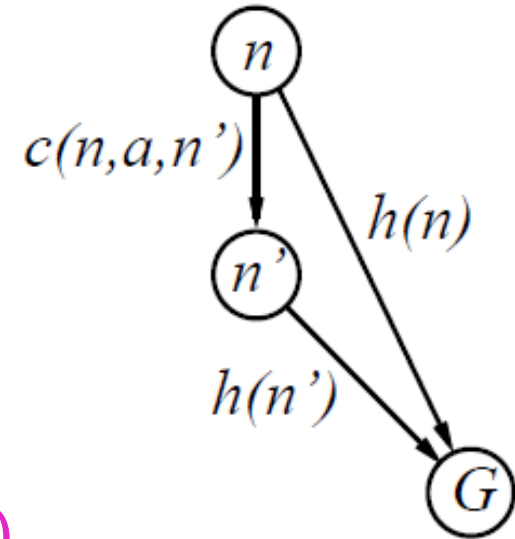
- A heuristic is monotonic if

$$h(n) \leq c(n, a, n') + h(n')$$

- If h is monotonic, we have

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) \\ &= f(n) \end{aligned}$$

- i.e., $f(n)$ is monotonic along any path



Triangle
Inequality

Effect of Heuristic Accuracy on Performance (1)

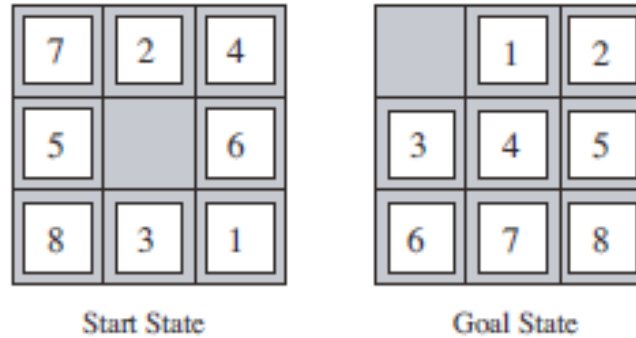
- ❑ Total number of nodes generated by A* search – N
- ❑ Solution depth is d
- ❑ Effective branching factor - b^* - branching factor of a uniform tree of depth d with $N + 1$ nodes

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

- ❑ What should be the value of b^* to perform search efficiently?

Effect of Heuristic Accuracy on Performance (2)

❑ 8-puzzle problem



❑ Two heuristics

- Number of misplaced tiles – h_1
- Total Manhattan distance - h_2

Effect of Heuristic Accuracy on Performance (3)

8-puzzle problem

	Search Cost (nodes generated)			Effective Branching Factor		
d	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	—	539	113	—	1.44	1.23
16	—	1301	211	—	1.45	1.25
18	—	3056	363	—	1.46	1.26
20	—	7276	676	—	1.47	1.27
22	—	18094	1219	—	1.48	1.28
24	—	39135	1641	—	1.48	1.26

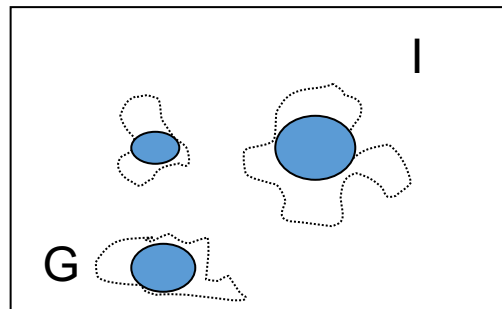
Figure 3.29 Comparison of the search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A^* algorithms with h_1 , h_2 . Data are averaged over 100 instances of the 8-puzzle for each of various solution lengths d .

Effect of Heuristic Accuracy on Performance (4)

- ❑ For two heuristics - h_1 and h_2
- ❑ If $h_2(n) \geq h_1(n), \forall n$
- ❑ Then h_2 dominates h_1
- ❑ *Domination* ~ efficiency of search – in terms of number of nodes expanded
 - h_2 will never expand more nodes than A^* using h_1 (except for some nodes with $f(n) = C^*$)

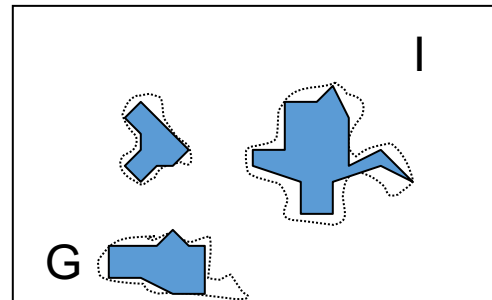
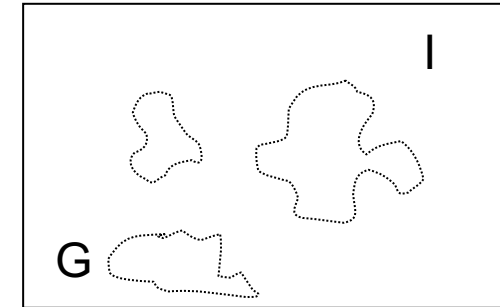
Generating Admissible Heuristic – Relaxed Problems

□ Shortest Path Problem on the plane



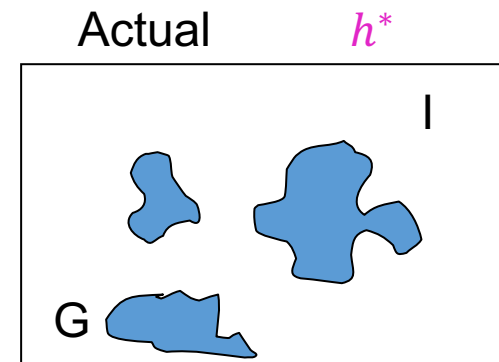
“circular abstraction”

h_C



“Polygonal abstraction”

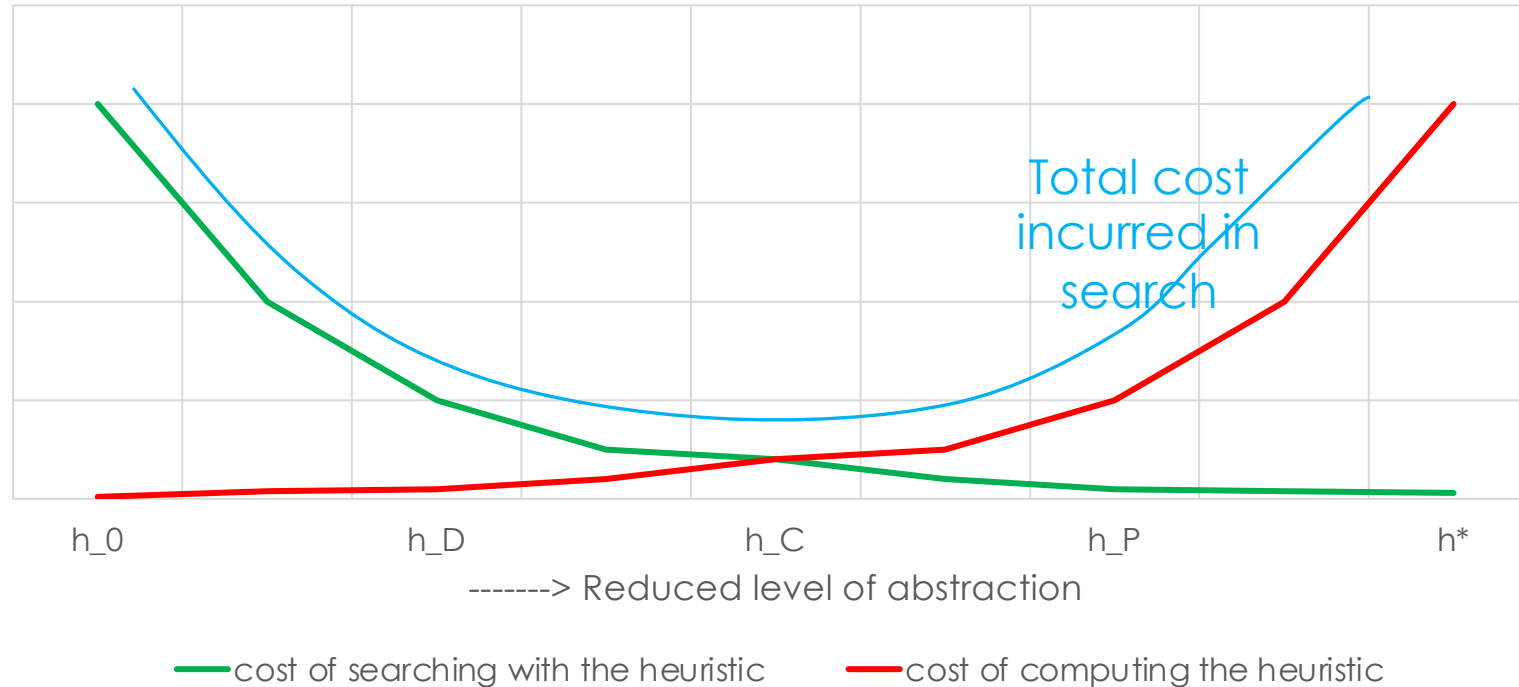
h_P



Actual

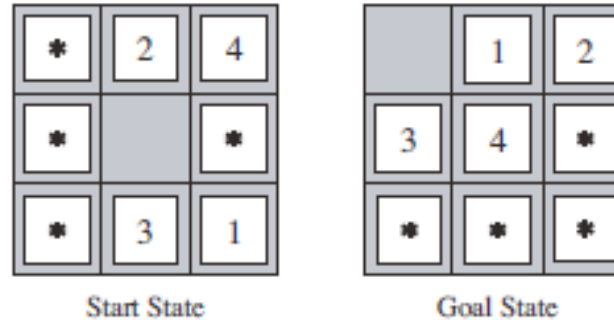
h^*

Heuristic Functions - Abstractions



Generating Admissible Heuristics – Pattern Databases

- ❑ Store the exact solution costs for every sub problem instance



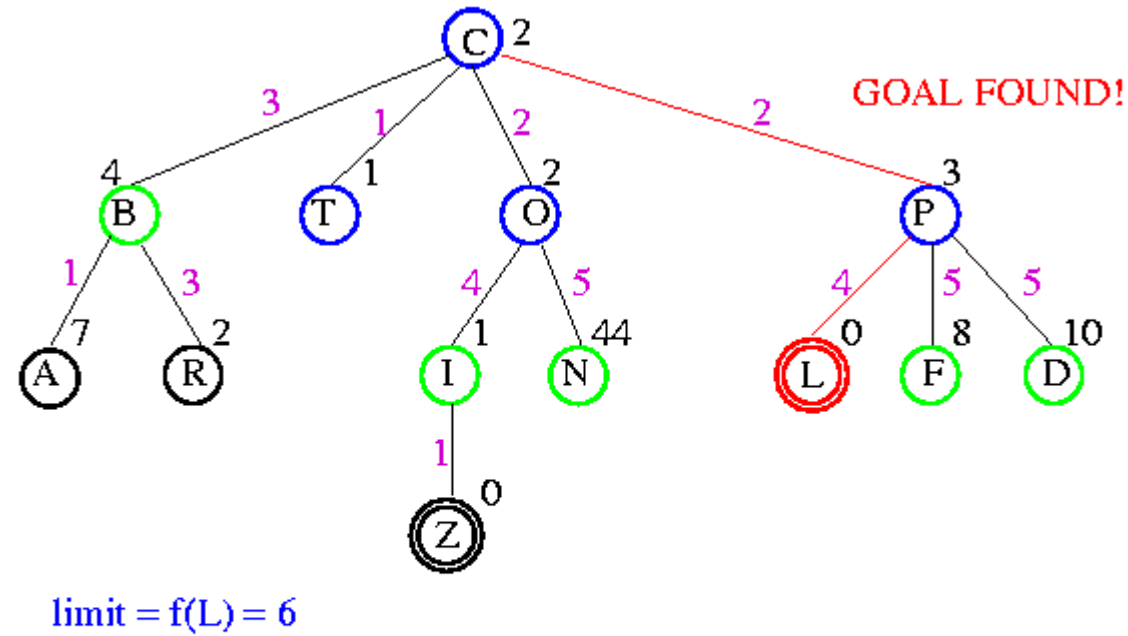
- ❑ Admissible heuristic will be cost for solving the corresponding sub problem.
 - Can be generated by working backwards from the goal state

Iterative Deepening A* (IDA*) Search

- ❑ Essentially IDDS, that uses f as the cost threshold, instead of depth.
- ❑ Implementation: add child to the queue if $f(child) < threshold$.

- Start with f cutoff equal to the f value of the root node
- Loop until solution is found
 - Generate and search all nodes whose f values are $\leq$ the current threshold
 - Use DFS to search the trees in each iteration
 - Keep track of the node which has the smallest f value that is $>$ the current threshold.
 - If a goal node is found terminate, else set the threshold to be next highest f value and loop back.

IDA* Search - Example



IDA* Search - Analysis

- ❑ Completeness: yes
- ❑ Optimality: yes
- ❑ Time Complexity: worst case: b^d
 - When all nodes have distinct f values
- ❑ Space Complexity: linear - bd

Additional Readings

- ❑ Recursive Best First Search
- ❑ Simplified memory-bounded A*
- ❑ Beam Search